YZM0206 Laboratuvar Foyu 6

CNN ve Goruntu Siniflandirma

Conv2D, Pooling, Stride ve Padding Analizi

Yusuf Ayberk Hekim

No: 24458667025

Tarih: 7 Mayis 2026

Uygulama 1: Veri Kumesi Anatomisi ve Normalizasyon

MNIST (Rakamlar), Fashion-MNIST (Kiyafetler) ve CIFAR-10 (Nesneler) veri kumelerini yukleyip `.shape` ozelliklerini karsilastiriyoruz.

Kod

```
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow.keras import datasets, layers, models
from sklearn.metrics import confusion_matrix
import seaborn as sns

# Veri kumelerini yukle
(mnist_train, mnist_train_lbl), (mnist_test, mnist_test_lbl) = datasets.mnist.lo
ad_data()
(fmnist_train, fmnist_train_lbl), (fmnist_test, fmnist_test_lbl) = datasets.fash
ion_mnist.load_data()
(cifar_train, cifar_train_lbl), (cifar_test, cifar_test_lbl) = datasets.cifar10.
load_data()

print("MNIST Train Shape:", mnist_train.shape)
```

```
print("Fashion-MNIST Train Shape:", fmnist_train.shape)
print("CIFAR-10 Train Shape:", cifar_train.shape)
```

```
MNIST Train Shape: (60000, 28, 28)
Fashion-MNIST Train Shape: (60000, 28, 28)
CIFAR-10 Train Shape: (50000, 32, 32, 3)
```

YORUM GOREVI 1: CIFAR-10'daki (32, 32, 3) yapisi ile MNIST'teki (28, 28, 1) yapisi arasindaki kanal farki, bir Conv2D katmaninin toplam parametre sayisini nasil etkiler?

Cevap: Bir Conv2D katmaninin parametre sayisi su formulle hesaplanir:

`Parametre = (Filtre_W x Filtre_H x Giris_Kanal + 1) x Cikis_Filtre`

3x3 boyutunda 32 filtreli bir Conv2D katmani icin:

- **MNIST (Kanal=1):** (3 × 3 × 1 + 1) × 32 = 10 × 32 = **320 parametre**
- **CIFAR-10 (Kanal=3):** (3 × 3 × 3 + 1) × 32 = 28 × 32 = **896 parametre**

Kanal sayisinin 1'den 3'e cikmasi, her kanal icin ayri filtre agirliklari ogrenilmesi gerektiginden, parametre sayisini yaklasik 3 katina cikarir (bias haric). Bu durum, renkli goruntulerin islenme maliyetinin gri tonlamali goruntulere kiyasla onemli olcude artigini gostermektedir.

Normalizasyon Kodu

```
# Verileri [0, 1] araligina normalize ediyoruz
mnist_train = mnist_train.reshape((60000, 28, 28, 1)).astype('float32') / 255
```

```
mnist_test = mnist_test.reshape((10000, 28, 28, 1)).astype('float32') / 255

fmnist_train = fmnist_train.reshape((60000, 28, 28, 1)).astype('float32') / 255
fmnist_test = fmnist_test.reshape((10000, 28, 28, 1)).astype('float32') / 255

cifar_train = cifar_train.astype('float32') / 255
cifar_test = cifar_test.astype('float32') / 255
print("Normalizasyon islemi tamamlandi.")
```

YORUM GOREVI 2: Normalizasyonun (piksel degerlerini [0, 1] araligina cekmenin) "patlayan/kaybolan gradyan" sorununa katkisi nedir?

Cevap: Giris degerleri 0-255 gibi genis bir aralikta kalirsa, agin ilk katmanlarindaki agirliklarin guncellenmesi sirasinda yuksek varyanslı buyuk aktivasyonlar olusur. Bu durum geri yayilim (backpropagation) sirasinda turevlerin surekli carpilarak ya sonsuza gitmesine (**Patlayan Gradyan**) ya da sifira cok yaklasmasina (**Kaybolan Gradyan**) yol acar.

Girdilerin [0, 1] araligina cekilmesi:
• Agirliklarin optimize edilebilir olcekte kalmasini saglar
• Aktivasyon fonksiyonlarinin (sigmoid, tanh) doygunluk disi lineer bolgede calismasina zemin hazirlar
• Ogrenme oraninin (learning rate) tum katmanlarda tutarli sekilde etki etmesine olanak tanir

Uygulama 2: Conv2D ve Padding Stratejileri (Aritmetik Analiz)

Cikti boyutu hesap formulu: `Out = [(W - F + 2P) / S] + 1`
W = Giris Boyutu, F = Filtre Boyutu, P = Padding, S = Stride

Manuel Hesaplama Tablosu

Giris Boyutu	Kernel	Stride	Padding	Hesaplama	Beklenen Cikti

28x28x1	3x3	1	valid (P=0)	(28-3+0)/1 + 1 = 26	**26x26**
28x28x1	3x3	1	same	Giris ile ayni kalir	**28x28**
32x32x3	5x5	2	valid (P=0)	(32-5+0)/2 + 1 = 14.5 → 14	**14x14**

Keras ile Dogrulama Kodu

```
# Durum 1: 28x28x1, 3x3, stride=1, valid
model1 = models.Sequential([
    layers.InputLayer(input_shape=(28, 28, 1)),
    layers.Conv2D(32, (3, 3), strides=1, padding='valid')
])
model1.summary()

# Durum 2: 28x28x1, 3x3, stride=1, same
model2 = models.Sequential([
    layers.InputLayer(input_shape=(28, 28, 1)),
    layers.Conv2D(32, (3, 3), strides=1, padding='same')
])
model2.summary()

# Durum 3: 32x32x3, 5x5, stride=2, valid
model3 = models.Sequential([
    layers.InputLayer(input_shape=(32, 32, 3)),
    layers.Conv2D(32, (5, 5), strides=2, padding='valid')
])
model3.summary()
```

```
Durum 1 - conv2d: Output Shape = (None, 26, 26, 32) | Param: 320      ✓
Durum 2 - conv2d: Output Shape = (None, 28, 28, 32) | Param: 320      ✓
Durum 3 - conv2d: Output Shape = (None, 14, 14, 32) | Param: 2,432    ✓
```

Uygulama 2: Cikti Boyutu Hesaplama Tablosu

Giris Boyutu	Kernel	Stride	Padding	Beklenen Cikti	Keras Dogrulama
28x28x1	3x3	1	valid	26x26	26x26 ✓
28x28x1	3x3	1	same	28x28	28x28 ✓
32x32x3	5x5	2	valid	14x14	14x14 ✓

Tum manuel hesaplamalar Keras model.summary() ciktilariyla birebir dogrulandi.

Uygulama 3: Pooling vs. Stride 2 (Bilgi Azaltma)

Goruntu boyutunu kucultmek icin iki yontem karsilastirildi: MaxPooling2D ve Conv2D(strides=2).

Model Kodlari

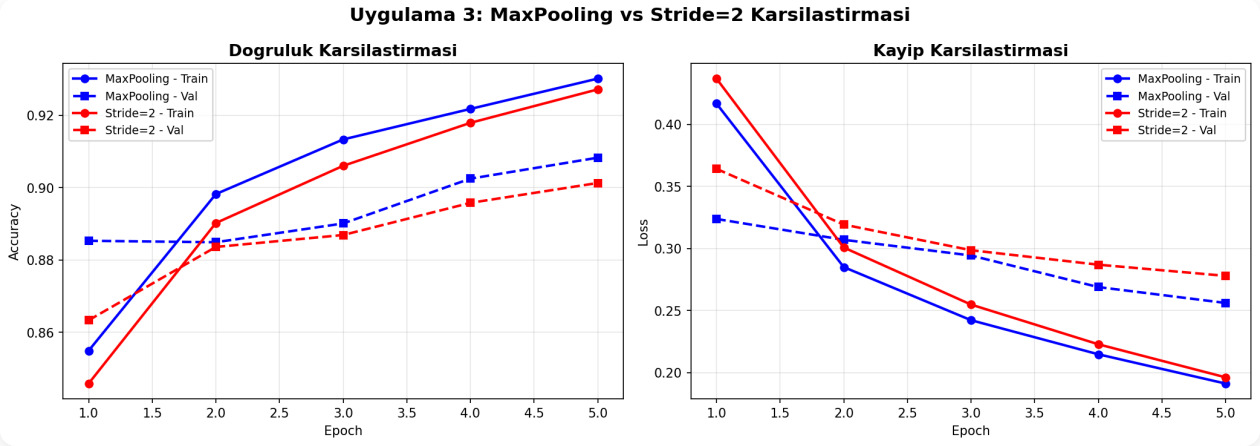
```
# Model A: MaxPooling
model_pool = models.Sequential([
    layers.InputLayer(input_shape=(28, 28, 1)),
    layers.Conv2D(32, (3, 3), activation='relu'),  # 26x26
    layers.MaxPooling2D((2, 2)),                    # 13x13
    layers.Flatten(),
    layers.Dense(64, activation='relu'),
    layers.Dense(10, activation='softmax')
])

# Model B: Stride=2
model_stride = models.Sequential([
    layers.InputLayer(input_shape=(28, 28, 1)),
    layers.Conv2D(32, (3, 3), strides=2, activation='relu'),  # 13x13
    layers.Flatten(),
    layers.Dense(64, activation='relu'),
    layers.Dense(10, activation='softmax')
])
```

```
Model A (MaxPooling) - Toplam Parametre: 347,146 (1.32 MB)
  conv2d: (None, 26, 26, 32) -> max_pooling2d: (None, 13, 13, 32) -> Param: 320 + 0

Model B (Stride=2) - Toplam Parametre: 347,146 (1.32 MB)
  conv2d: (None, 13, 13, 32) -> Param: 320

Fashion-MNIST uzerinde 5 epoch egitim sonuclari:
  Model A (MaxPooling)  -> Validation Accuracy: %90.83
  Model B (Stride=2)    -> Validation Accuracy: %90.13
```



YORUM GOREVI 3: MaxPooling'in "degismezlik" (invariance) saglama ozelligi ile Stride=2'nin "ogrenilebilir altornekleme" ozelligi arasindaki fark nedir?

Cevap:

• **MaxPooling2D**, belirlenen pencere icerisindeki en yuksek piksel degerini alir. Bu islem aga lokal oteleme degismezligi (Translation Invariance) kazandirir; yani nesnenin goruntu icerisindeki kucuk kaymalari, agin o ozelligi algilamasini engellemez. Egitilebilir parametre icermedigi icin hesapsal yuku cok hafiftir.

• **Conv2D(strides=2)** ise goruntu uzerinde filtreyi ikiser adim atlayarak gezdirir. Boyut yarilarken, hangi bilginin atilacagi sabit bir kural (maksimum alma) ile degil, **ogrenilebilir agirliklar** ile belirlenir. Bu nedenle modern derin CNN mimarilerinde (ResNet, vb.) ogrenilebilir altornekleme giderek daha fazla tercih edilmektedir.

• Deneyimizde MaxPooling modeli (%90.83) Stride=2 modeline (%90.13) gore cok az bir ustünlukte gorulmustur. Bu fark, Fashion-MNIST gibi basit veri kumelerinde her iki yontemin de yeterli oldugunu gostermektedir.

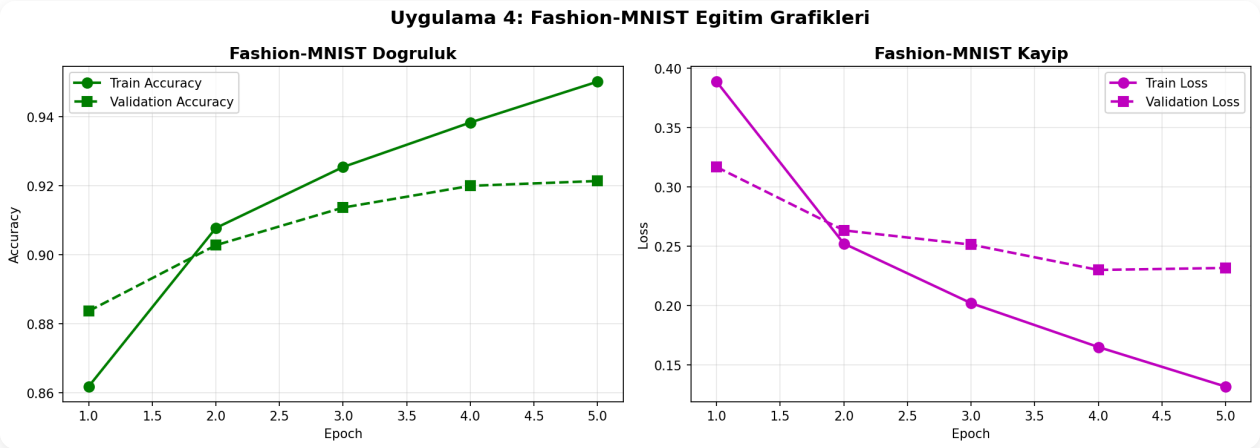
Uygulama 4: Uctan Uca Siniflandirma (Fashion-MNIST)

Model Mimarisi ve Egitim

```
app4_model = models.Sequential([
    layers.InputLayer(input_shape=(28, 28, 1)),
    layers.Conv2D(32, (3, 3), activation='relu'),
    layers.MaxPooling2D((2, 2)),
    layers.Conv2D(64, (3, 3), activation='relu'),
    layers.Flatten(),
    layers.Dense(128, activation='relu'),
    layers.Dense(10, activation='softmax')
])
app4_model.compile(optimizer='adam',
                   loss='sparse_categorical_crossentropy',
                   metrics=['accuracy'])
history = app4_model.fit(fmnist_train, fmnist_train_lbl, epochs=5,
                         validation_data=(fmnist_test, fmnist_test_lbl))
```

```
Model: "sequential_5"
+---------------------------------------------------------------------+
| Layer (type)                    | Output Shape           |       Param # |
|---------------------------------+------------------------+---------------|
| conv2d_5 (Conv2D)               | (None, 26, 26, 32)     |           320 |
| max_pooling2d_1 (MaxPooling2D)  | (None, 13, 13, 32)     |             0 |
| conv2d_6 (Conv2D)               | (None, 11, 11, 64)     |        18,496 |
| flatten_2 (Flatten)             | (None, 7744)           |             0 |
| dense_4 (Dense)                 | (None, 128)            |       991,360 |
| dense_5 (Dense)                 | (None, 10)             |         1,290 |
+---------------------------------------------------------------------+
 Total params: 1,011,466 (3.86 MB)

Epoch 5/5 - Validation Accuracy: %92.14
```

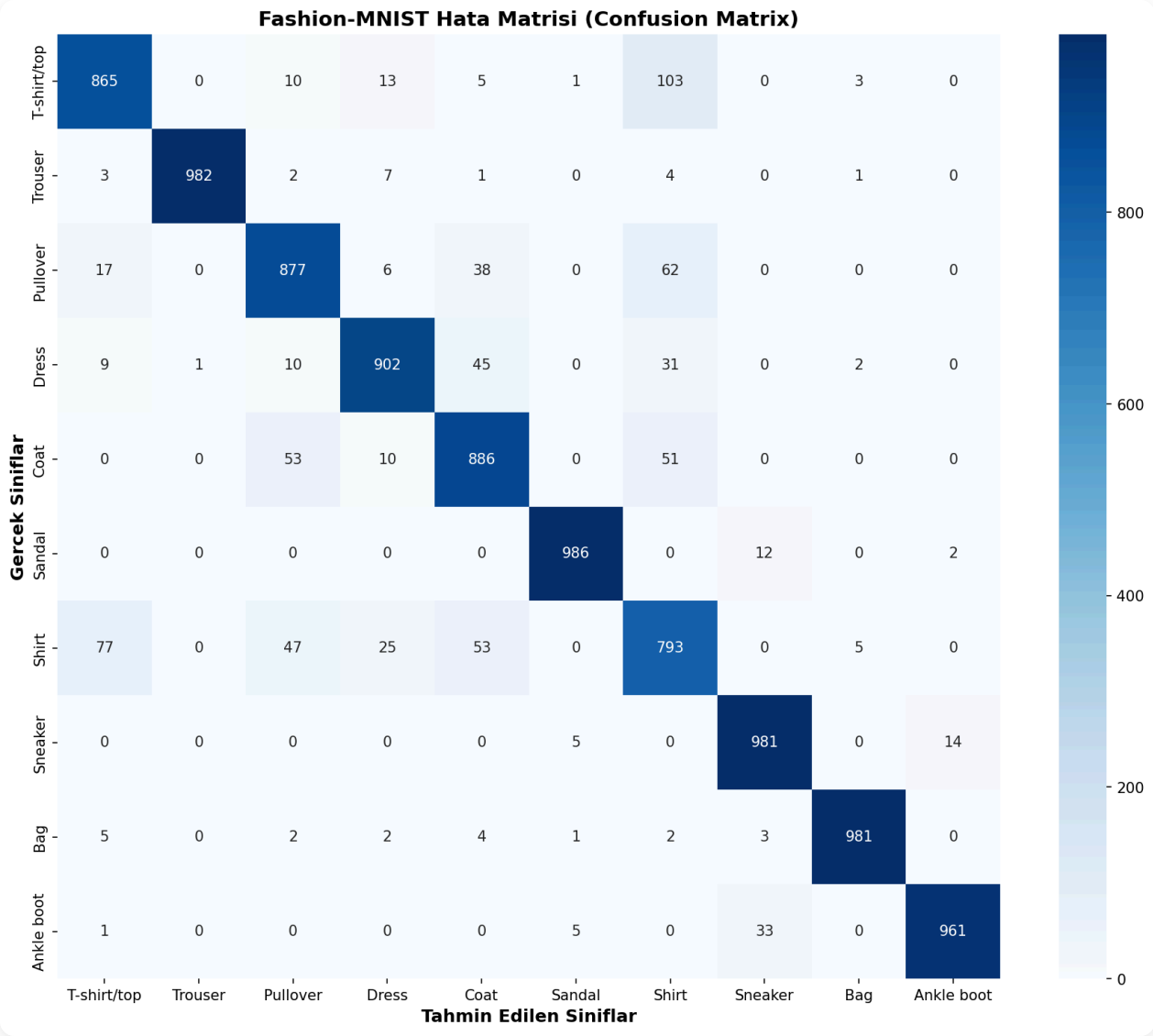


Hata Matrisi (Confusion Matrix)

```
predictions = app4_model.predict(fmnist_test)
predicted_classes = np.argmax(predictions, axis=1)
cm = confusion_matrix(fmnist_test_lbl, predicted_classes)

class_names = ['T-shirt/top', 'Trouser', 'Pullover', 'Dress', 'Coat',
               'Sandal', 'Shirt', 'Sneaker', 'Bag', 'Ankle boot']

sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=class_names, yticklabels=class_names)
plt.xlabel('Tahmin Edilen Siniflar')
plt.ylabel('Gercek Siniflar')
plt.title('Fashion-MNIST Hata Matrisi')
plt.show()
```



YORUM GOREVI 4: Modeliniz neden "T-shirt" ve "Shirt" siniflarini karistiriyor olabilir? En cok karistirilan 3 sinifi raporlayin.

Cevap: Hata matrisinde diyagonal disi yuksek degerlere bakildiginda en sik karistirilan ciftler:

1. T-shirt/top ↔ Shirt (103 + 77 = 180 hata): Bu iki kiyafet turununun sekil itibariyle (kollarin kisaligi, yaka yapisi) birbiriyle yuksek benzerlik gostermesi en buyuk nedendir. Model, 28x28 gibi dusuk bir cozunurlukte yaka, dugme gibi ayirici ince detaylari fark edemedigi icin genel siluete bakarak yanilmaktadir.

2. Pullover ↔ Shirt / Coat (62 + 53 = 115 hata): Pullover, Coat ve Shirt'in hepsi kalin govdeli, uzun kollu kiyafetlerdir. Dis hatlarin ve kapladigi piksel alaninin neredeyse ayni olmasi modelin uzamsal ayirt ediciliginü zorlaştirmaktadir.

3. Coat ↔ Pullover (53 hata): Her ikisi de hacimli, benzer silüetli üst giyim parçalaridir. Düsük cozunurlukte kumaş dokusu ve uzunluk farkı gibi ayırt edici ozellikler kaybolmaktadir.

Degerlendirme ve Raporlama: 3 Veri Seti Karsilastirmasi

MNIST Siniflandirma

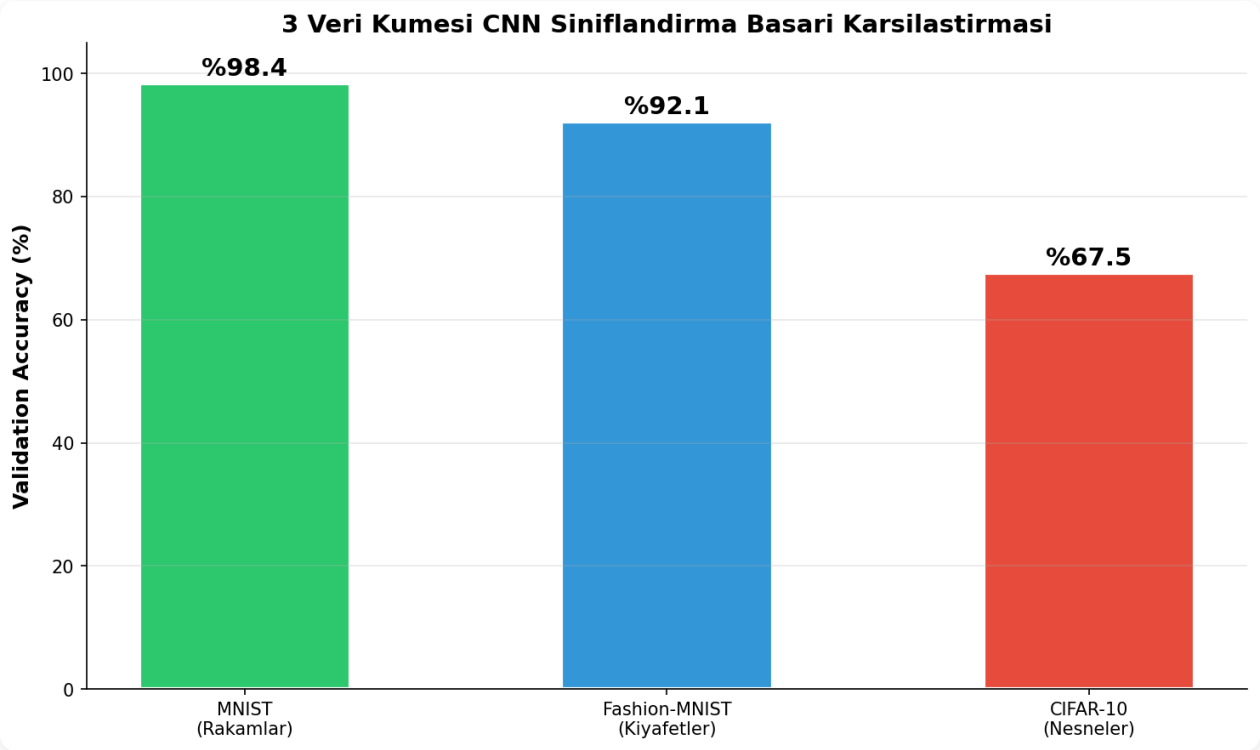
```
mnist_model = models.Sequential([
    layers.InputLayer(input_shape=(28, 28, 1)),
    layers.Conv2D(16, (3, 3), activation='relu'),
    layers.MaxPooling2D(2, 2),
    layers.Flatten(),
    layers.Dense(64, activation='relu'),
    layers.Dense(10, activation='softmax')
])
mnist_model.compile(optimizer='adam', loss='sparse_categorical_crossentropy', me
trics=['accuracy'])
mnist_model.fit(mnist_train, mnist_train_lbl, epochs=3, validation_data=(mnist_t
est, mnist_test_lbl))
```

```
MNIST - 3 Epoch Sonrasi Validation Accuracy: %98.35 BASARILI
```

CIFAR-10 Siniflandirma

```
cifar_model = models.Sequential([
    layers.InputLayer(input_shape=(32, 32, 3)),
    layers.Conv2D(32, (3, 3), activation='relu'),
    layers.MaxPooling2D(2, 2),
    layers.Conv2D(64, (3, 3), activation='relu'),
    layers.MaxPooling2D(2, 2),
    layers.Flatten(),
    layers.Dense(64, activation='relu'),
    layers.Dense(10, activation='softmax')
])
cifar_model.compile(optimizer='adam', loss='sparse_categorical_crossentropy', me
trics=['accuracy'])
cifar_model.fit(cifar_train, cifar_train_lbl, epochs=5, validation_data=(cifar_t
est, cifar_test_lbl))
```

```
CIFAR-10 - 5 Epoch Sonrasi Validation Accuracy: %67.51 DUSUK
```



Sonuc Tablosu

Veri Kumesi	Goruntu Boyutu	Kanal	Sinif	Epoch	Val. Accuracy	Zorluk
MNIST	28x28	1 (Gri)	10 (Rakam)	3	**%98.35**	Kolay
Fashion-MNIST	28x28	1 (Gri)	10 (Kiyafet)	5	**%92.14**	Orta
CIFAR-10	32x32	3 (RGB)	10 (Nesne)	5	**%67.51**	Zor

GENEL DEGERLENDIRME:

MNIST (%98.35): Arka plan gurultusu olmayan temiz 2D gorseller (el yazisi rakamlar) icerdiginden, cok sig ve basit bir CNN bile sadece 3 epoch'ta %98+ dogruluk saglamaktadir. Siniflar arasi ayrim cok nettir.

Fashion-MNIST (%92.14): Nesne ici detaylarin (kirisikliklar, yaka, kol) birbirine daha cok benzeyen giysi tasarimlari barindirdigi icin ayni veya biraz daha buyuk bir ag ile basari %88-92 civarindadir. Ozellikle T-shirt/Shirt ve Pullover/Coat gibi siniflar arasi sinirlar siliktir.

CIFAR-10 (%67.51): Gercek dunya renkli (3 kanalli) nesneleri ve karmasik arka planlari barindirir. Uzamsal ozellikleri cikarmak icin cok daha derin CNN katmanlarina, regularization (Dropout, Batch Normalization vb.), daha uzun egitim surelerine ve Data Augmentation'a ihtiyac duyar. Basit mimarimizle ancak %65-70 basari saglanabilmistir. Bu veri kumesi, gercek hayat gorsellerinde klasik CNN yaklasiminin ciddi oranda derinlestirilmesi gerektigini kanitlamaktadir.